

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 1
MODUL 14
“REVIEW ALGORITMA DAN PEMROGRAMAN 1”



DISUSUN OLEH:

Sofwan Nuha Al Faruq | 109082500032 | IF-13 02

DOSEN:

Wahyu Andi Saputra, S.Pd., M.Eng.

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2024/2025

1. Latihan 1: Hercules 1 (Selection Sort)

Source Code:

```
package main
import "fmt"

func main() {
    var n, m int
    fmt.Scan(&n)
    for i := 0; i < n; i++ {
        fmt.Scan(&m)
        arr := make([]int, m)
        for j := 0; j < m; j++ {
            fmt.Scan(&arr[j])
        }

        for j := 0; j < m-1; j++ {
            idx_min := j
            for k := j + 1; k < m; k++ {
                if arr[k] < arr[idx_min] {
                    idx_min = k
                }
            }
            arr[j], arr[idx_min] = arr[idx_min], arr[j]
        }

        for j := 0; j < m; j++ {
            fmt.Print(arr[j], " ")
        }
        fmt.Println()
    }
}
```

Output:

```
1 2 7 9 13
15 27 39 75 133 189
1 4 9
```

Deskripsi Program: Program ini membaca jumlah daerah kerabat dan mengumpulkan input nomor rumah secara berurutan di tiap daerah ke dalam sebuah *array*. Setelah kumpulan data terkumpul, program akan menyusun nomor-nomor rumah tersebut secara terurut membesar (*ascending*) menggunakan algoritma *Selection Sort*.

Pada algoritma ini, program mencari nilai rumah yang paling kecil pada rentang data yang tersisa atau belum terurut, lalu menukarnya (*swap*) dengan data yang berada pada posisi paling ujung kiri dari rentang data

tersebut. Begitu seluruh elemen pada suatu daerah telah selesai diurutkan membesar, deretan nomor rumah dicetak per baris sebelum program memproses daerah selanjutnya.

2. Latihan 2: Kerabat Dekat (Selection Sort Ganjil Genap)

Source Code:

```
package main
import "fmt"

func main() {
    var n, m, x int
    fmt.Scan(&n)
    for i := 0; i < n; i++ {
        fmt.Scan(&m)
        ganjil := []int{}
        genap := []int{}
        for j := 0; j < m; j++ {
            fmt.Scan(&x)
            if x%2 != 0 {
                ganjil = append(ganjil, x)
            } else {
                genap = append(genap, x)
            }
        }

        for j := 0; j < len(ganjil)-1; j++ {
            idx := j
            for k := j + 1; k < len(ganjil); k++ {
                if ganjil[k] < ganjil[idx] { idx = k }
            }
            ganjil[j], ganjil[idx] = ganjil[idx], ganjil[j]
        }

        for j := 0; j < len(genap)-1; j++ {
            idx := j
            for k := j + 1; k < len(genap); k++ {
                if genap[k] > genap[idx] { idx = k }
            }
            genap[j], genap[idx] = genap[idx], genap[j]
        }

        for _, v := range ganjil { fmt.Print(v, " ") }
        for _, v := range genap { fmt.Print(v, " ") }
        fmt.Println()
    }
}
```

```
}  
}
```

Output:

```
1 13 12 8 2  
15 27 39 75 133 189  
8 4 2
```

Deskripsi Program: Sesuai dengan petunjuk studi kasus, program memisahkan bilangan ganjil dan genap ke dalam dua *array* yang berbeda pada saat membaca data masukan. Strategi pemisahan ini mempermudah proses pengurutan karena kondisi yang diminta pada luaran saling berkebalikan: nomor rumah sisi kiri jalan (ganjil) diurutkan membesar (*ascending*), sedangkan nomor rumah sisi kanan jalan (genap) diurutkan mengecil (*descending*).

Masing-masing array tersebut kemudian diurutkan secara terpisah menggunakan modifikasi perulangan algoritma *Selection Sort*. Pada saat mencetak hasilnya, program melakukan perulangan untuk mencetak semua nilai dari kumpulan elemen ganjil lebih dulu secara berurutan, lalu segera dilanjutkan mencetak semua nilai genapnya pada baris yang sama.

3. Latihan 3: Median (Insertion/Selection Sort Dinamis)**Source Code:**

```
package main  
import "fmt"  
  
func main() {  
    var x int  
    arr := []int{}  
  
    for {  
        fmt.Scan(&x)  
        if x == -5313 {  
            break  
        } else if x != 0 {  
            arr = append(arr, x)  
        } else {  
            for j := 1; j < len(arr); j++ {  
                temp := arr[j]  
                k := j  
                for k > 0 && temp < arr[k-1] {  
                    arr[k] = arr[k-1]  
                    k--  
                }  
                arr[k] = temp  
            }  
        }  
    }  
}
```



```

    }

    for i := 1; i < len(arr); i++ {
        temp := arr[i]
        j := i
        for j > 0 && temp < arr[j-1] {
            arr[j] = arr[j-1]
            j--
        }
        arr[j] = temp
    }

    for _, v := range arr {
        fmt.Print(v, " ")
    }
    fmt.Println()

    isKonsisten := true
    jarak := arr[10] - arr
    for i := 2; i < len(arr); i++ {
        if arr[i]-arr[i-1] != jarak {
            isKonsisten = false
            break
        }
    }

    if isKonsisten {
        fmt.Printf("Data berjarak %d\n", jarak)
    } else {
        fmt.Println("Data berjarak tidak tetap")
    }
}

```

Output:

1 7 13 19 25 31 37 43

Data berjarak 6

Deskripsi Program: Kode program ini dirancang untuk menerima sekumpulan bilangan bulat dan hanya merekam bilangan bertipe non-negatif masuk ke memori *array*, di mana pembacaannya secara otomatis diakhiri oleh sebuah bilangan negatif. Algoritma *Insertion Sort* dipasang untuk mengurutkan kumpulan elemen array secara membesar, yang bekerja dengan membuat ruang kosong dan menyisipkan suatu nilai ke posisi yang seharusnya dengan metode pencarian sekuensial ke sebelah kiri. Setelah array diurutkan dan keseluruhan isi elemennya dicetak di baris pertama luaran, program melakukan tahapan validasi dengan memeriksa konsistensi pola selisih antar angkanya. Jika selisih antar angka pertama dan kedua selalu konstan nilainya ketika diujikan di setiap elemen yang bersisian hingga ke pangkal array,

program mencetak status berupa "Data berjarak x", sebaliknya jika ada satu jarak saja yang melenceng maka statusnya dicetak sebagai "Data berjarak tidak tetap".

5. Latihan 5: Program Perpustakaan (Insertion Sort pada Data Terstruktur)

Source Code:

```
package main
import "fmt"

const nMax int = 7919
type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating    int
}
type DaftarBuku [nMax]Buku

func DaftarkanBuku(pustaka *DaftarBuku, n *int) {
    fmt.Scan(n)
    for i := 0; i < *n; i++ {
        fmt.Scan(&pustaka[i].id, &pustaka[i].judul, &pustaka[i].penulis,
            &pustaka[i].penerbit, &pustaka[i].eksemplar, &pustaka[i].tahun,
            &pustaka[i].rating)
    }
}

func UrutBuku(pustaka *DaftarBuku, n int) {
    for i := 1; i < n; i++ {
        temp := pustaka[i]
        j := i
        for j > 0 && temp.rating > pustaka[j-1].rating {
            pustaka[j] = pustaka[j-1]
            j--
        }
        pustaka[j] = temp
    }
}

func CetakTerfavorit(pustaka DaftarBuku, n int) {
    fmt.Println(pustaka.judul, pustaka.penulis, pustaka.penerbit, pustaka.tahun)
}

func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    limit := 5
```



```

        if n < 5 { limit = n }
        for i := 0; i < limit; i++ {
            fmt.Println(pustaka[i].judul)
        }
    }

func CariBuku(pustaka DaftarBuku, n int, r int) {
    kiri, kanan := 0, n-1
    ketemu := false
    for kiri <= kanan && !ketemu {
        tengah := (kiri + kanan) / 2
        if pustaka[tengah].rating == r {
            fmt.Println(pustaka[tengah].judul, pustaka[tengah].penulis,
                pustaka[tengah].penerbit, pustaka[tengah].tahun,
                pustaka[tengah].eksemplar, pustaka[tengah].rating)
            ketemu = true
        } else if r > pustaka[tengah].rating {
            kanan = tengah - 1
        } else {
            kiri = tengah + 1
        }
    }
    if !ketemu { fmt.Println("Tidak ada buku dengan rating seperti itu") }
}

func main() {
    var pustaka DaftarBuku
    var n, ratingCari int
    DaftarkanBuku(&pustaka, &n)
    UrutBuku(&pustaka, n)
    CetakTerfavorit(pustaka, n)
    Cetak5Terbaru(pustaka, n)
    fmt.Scan(&ratingCari)
    CariBuku(pustaka, n, ratingCari)
}

```

Output: *(Luaran menyesuaikan format data dinamis tanpa test-case konkrit di modul)*

[Data Atribut Buku Terfavorit]

[Data Judul 5 Buku Tertinggi 1]

[Data Judul 5 Buku Tertinggi 2]

[Data Judul 5 Buku Tertinggi 3]

[Data Judul 5 Buku Tertinggi 4]

[Data Judul 5 Buku Tertinggi 5]

[Keterangan Buku Sesuai Rating Dicari / Status Tidak Ada]

Deskripsi Program: Sistem manajemen perpustakaan ini diimplementasikan menggunakan arsitektur tipe data bentukan (`struct Buku`) yang mampu membungkus ragam atribut lapangan seperti id, judul, penulis, penerbit, tahun, eksemplar, hingga nilai rating dalam satu kesatuan koleksi array yang sanggup menampung hingga 7919 data. Agar kodenya modular, program dipecah ke dalam lima *procedure* subprogram yang menjalankan status awal (*I.S.*) dan tujuan luaran akhir (*F.S.*) yang berbeda.

Tulang punggung manipulasi utama berada pada fungsi `UrutBuku`, yang bertugas menata *struct* buku secara menurun (*descending*) hanya berdasarkan *field* nilai *rating*-nya dengan memanfaatkan implementasi algoritma *Insertion Sort*. Begitu data buku berhasil terurut sedemikian rupa, prosedur pelaporan *CetakTerfavorit* dan *Cetak5Terbaru* dapat langsung memanggil isi elemen teratas, sementara pelacakan rating melalui mekanisme fungsi `CariBuku` dapat memanfaatkan algoritma *Binary Search* secara efisien dan konsisten